

RAPPORT DE PROJET DE MATHÉMATIQUES

Décomposition LU par blocs et résolution de système linéaire

Étudiants

Manh Hung	NGUYEN	mnguy00@insa-rouen.fr
Tan Minh Duy	NGO	tn00@insa-rouen.fr

22 novembre 2025

Encadré par

Bernard GLEYSE

Synthèse du projet

Titre du projet : Résolution numérique de systèmes linéaires par factorisation LU pour matrices tridiagonales et tridiagonales par blocs.

Résumé : Ce projet a pour objectif l'étude et l'implémentation de méthodes numériques pour la résolution de grands systèmes linéaires. Nous nous sommes intéressés d'abord au cas des matrices tridiagonales scalaires, puis au cas plus complexe des matrices tridiagonales par blocs. La factorisation LU a été programmée en langage C, suivie des étapes de descente et de remontée pour obtenir la solution numérique du système.

Les résultats obtenus montrent que la méthode LU fonctionne très bien lorsque la matrice est bien conditionnée, comme dans les tests dits "étude". Cependant, l'augmentation de la taille du système et l'utilisation de seconds membres généraux peuvent conduire à une amplification des erreurs numériques. Dans la partie blocs, nous avons observé une stabilité parfaite pour un système de taille modérée, puis l'apparition d'une instabilité lors d'un test plus général, révélant les limites de la factorisation LU pour les matrices de grande dimension.

Ce travail nous a permis de mettre en pratique les notions vues en cours, de comprendre l'influence du conditionnement sur la précision des résultats, et de prendre conscience de l'importance de la stabilité numérique dans les méthodes directes de résolution de systèmes linéaires.

Mots clés : factorisation LU ; matrices tridiagonales ; matrices par blocs ; conditionnement.

Table des matières

Synthèse du projet	1
Introduction	3
1 Présentation du problème mathématique	4
2 Méthode numérique et implémentation	8
3 Resultats numériques et analyse	10
Conclusion	16

Introduction

Dans ce projet MMSN, nous étudions la décomposition LU par blocs d'une matrice tridiagonale par blocs et son utilisation pour la résolution d'un système linéaire.

Ce type de problème apparaît souvent dans les calculs scientifiques et en ingénierie, par exemple lors de la discrétisation d'équations différentielles partielles. La matrice obtenues possèdent alors une structure particulière qui peut être exploitée pour réduire le temps de calcul et la mémoire nécessaire.

L'objectif principal de ce travail est de comprendre le principe de la factorisation LU par blocs, de l'implémenter en langage C sous Linux, puis de l'utiliser pour résoudre un système linéaire $Ax = b$. Nous comparerons ensuite nos résultats avec ceux obtenus à l'aide d'une bibliothèque numérique standard, afin d'évaluer la précision et la stabilité de la méthode.

Ce rapport présente d'abord le cadre mathématique du problème et les formules utilisées pour la factorisation. Ensuite, nous décrivons la méthode numérique choisie et les étapes de programmation. Enfin, nous analysons les résultats numériques obtenus et concluons sur l'efficacité et les limites de cette approche.

1 Présentation du problème mathématique

Dans ce projet, nous intéressons à la factorisation LU d'une matrice tridiagonale par blocs et à la résolution du système linéaire associé. Une matrice tridiagonale est une matrice carrée dont les éléments non nuls se trouvent uniquement sur la diagonale principale et sur les deux diagonales voisines.

Matrice tridiagonale scalaire

Dans le cas simple, La matrice A de taille $N \times N$ peut s'écrire sous la forme :

$$A = \begin{bmatrix} b_1 & c_1 & 0 & \cdots & 0 \\ a_2 & b_2 & c_2 & \ddots & \vdots \\ 0 & a_3 & b_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c_{N-1} \\ 0 & \cdots & 0 & a_N & b_N \end{bmatrix}$$

où a_i, b_i et c_i sont les coefficients des diagonales inférieure, principale et supérieure.

L'objectif de la décomposition LU est d'écrire A sous la forme : $A = LU$ où L est une matrice triangulaire inférieure, avec des 1 sur la diagonale principale et U est une matrice triangulaire supérieure. On cherche à déterminer les éléments de L et de U à partir des coefficients a_i, b_i, c_i de A .

Par identification des coefficients de A , on vérifie que L et U sont bidiagonales sous la forme :

$$L = \begin{bmatrix} 1 & 0 & 0 & \cdots & 0 \\ \ell_2 & 1 & 0 & \cdots & 0 \\ 0 & \ell_3 & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & \ell_N & 1 \end{bmatrix}, \quad U = \begin{bmatrix} d_1 & c_1 & 0 & \cdots & 0 \\ 0 & d_2 & c_2 & \ddots & \vdots \\ 0 & 0 & d_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & c_{N-1} \\ 0 & \cdots & 0 & 0 & d_N \end{bmatrix}.$$

Coefficients de la sous-diagonale

Le terme $A(i, i-1)$ se calcule ainsi :

$$A(i, i-1) = L(i, i-1) \times U(i-1, i-1) = \ell_i d_{i-1}$$

Comme on sait que $A(i, i - 1) = a_i$, on en déduit :

$$\ell_i = \frac{a_i}{d_{i-1}}$$

Coefficients de la sur-diagonale

Le terme $A(i, i + 1)$ vaut :

$$A(i, i + 1) = L(i, i) \times U(i, i + 1) = c_i$$

Donc $u_i = c_i$

Coefficients de la diagonale principale

Enfin :

$$A(i, i) = L(i, i - 1)U(i - 1, i) + L(i, i)U(i, i) = \ell_i c_{i-1} + d_i$$

En identifiant avec $b_i = A(i, i)$, on obtient :

$$d_i = b_i - \ell_i c_{i-1}$$

En regroupant les résultats précédents, on obtient le système de formules suivant :

$$\begin{cases} d_1 = b_1, \\ u_1 = c_1, \\ \ell_i = \frac{a_i}{d_{i-1}}, \quad \text{pour } 2 \leq i \leq N \\ d_i = b_i - \ell_i c_{i-1}, \quad \text{pour } 2 \leq i \leq N \end{cases}$$

Une fois les matrices L et U connues, la résolution du système $Ax = b$ se fait en deux étapes :

— **Descente** : on résout $Ly = b$, donc :

$$\begin{cases} y_1 = b_1, \\ y_i = b_i - \ell_i y_{i-1}, \quad 2 \leq i \leq N. \end{cases}$$

— **Remontée** : on résout ensuite $Ux = y$, donc :

$$\begin{cases} x_N = \frac{y_N}{d_N}, \\ x_i = \frac{y_i - c_i x_{i+1}}{d_i}, \quad (N-1) \geq i \geq 1. \end{cases}$$

Cette méthode est très efficace. Le nombre d'opérations est donc :

- Pour la factorisation LU : $(N-1)$ additions, multiplications et divisions.
- Pour la descente et la remontée : chacune a $(N-1)$ opérations.

Le nombre total d'opérations est donc proportionnel à N , donc la complexité est en $O(N)$. Cette efficacité est bien supérieure à la méthode classique comme méthode de Gausse, dont la complexité est en $O(N^3)$.

Matrice tridiagonale par blocs

Dans le cadre de ce projet, on généralise ces formules au cas par blocs. Au lieu de contenir des nombres scalaires sur les diagonales, elle contient des sous-matrices carrées, appelées blocs.

La matrice A s'écrit sous la forme :

$$A = \begin{bmatrix} B_1 & C_1 & 0 & \cdots & 0 \\ A_2 & B_2 & C_2 & \ddots & \vdots \\ 0 & A_3 & B_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & C_{N-1} \\ 0 & \cdots & 0 & A_N & B_N \end{bmatrix}$$

où A_i, B_i, C_i sont des matrices carrées de taille $n \times n$ appelées blocs, les autres cases contiennent des matrices nulles. Ainsi, A est une matrice carrée de taille totale $nN \times nN$.

Comme dans le cas scalaire, on cherche à écrire : $A = LU$, où L et U sont également des matrices tridiagonales par blocs.

Elles ont la forme suivante :

$$L = \begin{bmatrix} I & 0 & 0 & \cdots & 0 \\ L_2 & I & 0 & \cdots & 0 \\ 0 & L_3 & I & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & L_N & I \end{bmatrix}, \quad U = \begin{bmatrix} U_1 & C_1 & 0 & \cdots & 0 \\ 0 & U_2 & C_2 & \ddots & \vdots \\ 0 & 0 & U_3 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & C_{N-1} \\ 0 & \cdots & 0 & 0 & U_N \end{bmatrix}.$$

où I est la matrice identité de taille $n \times n$ et L_i, U_i sont des matrices carrées $n \times n$.

Les équations sont analogues au cas scalaire :

$$\begin{cases} U_1 = B_1, \\ L_i = A_i U_{i-1}^{-1}, \quad \text{pour } 2 \leq i \leq N, \\ U_i = B_i - L_i C_{i-1}, \quad \text{pour } 2 \leq i \leq N. \end{cases}$$

La différence importante est que les divisions deviennent des inversions de matrices blocs. Autrement dit, pour calculer L_i , on doit inverser la matrice U_{i-1} .

Une fois la factorisation $A = LU$ obtenue, on résout le système par deux étapes :

— **Descente** : on résout $LY = B$. Par blocs :

$$\begin{cases} Y_1 = B_1, \\ Y_i = B_i - L_i Y_{i-1}, \quad \text{pour } 2 \leq i \leq N. \end{cases}$$

— **Remontée** : on résout ensuite $UX = Y$, soit :

$$\begin{cases} X_N = U_N^{-1} Y_N, \\ X_i = U_i^{-1} (Y_i - C_i X_{i+1}), \quad \text{pour } (N-1) \geq i \geq 1. \end{cases}$$

2 Méthode numérique et implémentation

Cas scalaire : matrice tridiagonale classique

Dans la version scalaire, la matrice A possède trois diagonales : diagonale principale b_i , sous-diagonale a_i et sur-diagonale c_i . La factorisation conduit à :

- L : diagonale principale égale à 1, sous-diagonale contenant les coefficients ℓ_i ,
- U : diagonale principale contenant les coefficients d_i , et sur-diagonale identique aux c_i d'origine.

Les coefficients sont calculés de manière récursive :

$$\ell_i = \frac{a_i}{d_{i-1}}, \quad d_i = b_i - \ell_i c_{i-1}.$$

Cette procédure nécessite uniquement des opérations sur les diagonales, ce qui garantit une complexité linéaire $O(N)$, très avantageuse pour des systèmes de grande dimension.

L'implémentation en langage C reflète directement cette structure : les tableaux `diag_inf`, `diag`, `diag_sup`, `l` et `d` stockent respectivement a_i , b_i , c_i , ℓ_i et d_i . Une fois la factorisation obtenue, la descente et la remontée se font simplement par des boucles itératives calculant successivement les composantes du vecteur y puis x .

Cas par blocs : matrice tridiagonale par blocs

Lorsque la matrice A est constituée de blocs carrés $n \times n$, la méthode précédente se généralise naturellement. On travaille alors avec trois familles de blocs : blocs diagonaux B_i , blocs sous-diagonaux A_i et blocs sur-diagonaux C_i .

La factorisation $A = LU$ garde la même structure, mais la construction se fait bloc par bloc. Pour $i \geq 2$, on calcule :

$$L_i = A_i(U_{i-1})^{-1}, \quad U_i = B_i - L_i C_{i-1}.$$

Contrairement au cas scalaire, on doit manipuler des matrices complètes au lieu de simples valeurs. Cela implique l'utilisation d'opérations de matrice : multiplication de matrices, soustraction de matrices et inversion matricielle. Toutes ces opérations sont regroupées dans un fichier d'outils `matrices_tools.c` afin de rendre le code plus lisible et modulaire.

La résolution du système factorisé suit le même principe que précédemment :

- Descente : $y_1 = b_1$, puis $y_i = b_i - L_i y_{i-1}$,
- Remontée : $x_N = (U_N)^{-1} y_N$, puis $x_i = (U_i)^{-1} (y_i - C_i x_{i+1})$.

Ainsi, l'algorithme généralise fidèlement la méthode scalaire, mais en remplaçant les produits et divisions scalaires par des opérations matricielles.

Choix d'implémentation

Plusieurs choix ont été faits pour assurer efficacité et lisibilité :

- Stockage optimisé en utilisant uniquement les blocs utiles (structure tridiagonale respectée),
- Gestion séparée des fichiers d'entrée et de sortie afin d'adapter facilement les tests numériques,
- Modularisation du calcul bloc via un fichier d'outils (`matrices_tools.c`),
- Complexité linéaire en N (scalaire comme blocs).

Le passage à l'échelle est assuré tant que les blocs diagonaux U_i restent inversibles. Lorsque leur conditionnement se détériore (cas de grandes matrices Laplaciennes), la méthode peut devenir instable numériquement, nous aborderons dans la section suivante.

Enfin, afin d'analyser plus précisément la stabilité numérique des méthodes étudiées, nous avons également calculé le conditionnement des matrices utilisées dans les expériences. Plus une matrice est mal conditionnée, plus le résultat de la résolution $Ax = b$ est sensible aux erreurs d'arrondi et aux perturbations.

Pour cette raison, nous avons développé un fichier séparé permettant d'estimer le conditionnement en norme 2 des matrices tridiagonales, grâce à la fonction `numpy.linalg.cond`.

,

3 Resultats numériques et analyse

Dans cette section, nous présentons et analysons les résultats obtenus lors de l'application de la factorisation LU pour la résolution des systèmes tridiagonaux, dans les deux scalaire et blocs. Les fichiers de données utilisés sont fournis par le sujet du projet, et les solutions numériques sont comparées à la solution $x = (1, 1, \dots, 1)^T$. L'erreur calculée est la norme $\|X - x_{\text{exact}}\|_2$.

Cas scalaire

Test étude - matrice de diffusion, $N = 6$

Dans ce premier test scalaire, nous considérons une matrice de diffusion tridiagonale de taille $N = 6$. La matrice A est donnée par :

$$A = \begin{pmatrix} 2 & -1 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 \\ 0 & 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & 0 & -1 & 2 \end{pmatrix}.$$

Le vecteur second membre b est calculé pour ce test est

$$b = (1, 0, 0, 0, 0, 1)^T.$$

Le conditionnement en norme 2 de cette matrice a été estimé numériquement à

$$K_2(A) \approx 19,195669358089216,$$

ce qui reste une valeur modérée : le système n'est pas parfaitement conditionné, mais il n'est pas non plus fortement mal conditionné.

La factorisation LU obtenue numériquement a la forme suivante. La matrice triangulaire inférieure L possède une diagonale principale égale à 1, et une sous-diagonale

contenant des coefficients décroissants :

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ -0,5 & 1 & 0 & 0 & 0 & 0 \\ 0 & -\frac{2}{3} & 1 & 0 & 0 & 0 \\ 0 & 0 & -0,75 & 1 & 0 & 0 \\ 0 & 0 & 0 & -0,8 & 1 & 0 \\ 0 & 0 & 0 & 0 & -\frac{5}{6} & 1 \end{pmatrix},$$

tandis que la matrice triangulaire supérieure U reste tridiagonale, avec des valeurs positives sur la diagonale et -1 sur la sur-diagonale :

$$U = \begin{pmatrix} 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & 1,5 & -1 & 0 & 0 & 0 \\ 0 & 0 & \frac{4}{3} & -1 & 0 & 0 \\ 0 & 0 & 0 & 1,25 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1,2 & -1 \\ 0 & 0 & 0 & 0 & 0 & \frac{7}{6} \end{pmatrix}.$$

À partir de cette factorisation, le programme résout d'abord le système triangulaire inférieur $Ly = b$, puis le système triangulaire supérieur $Ux = y$. La solution numérique obtenue est

$$X = (1, 1, 1, 1, 1, 1)^T.$$

L'erreur par rapport au vecteur $\mathbf{1}$ est

$$\|X - \mathbf{1}\|_2 = 0,0000000000000000.$$

On observe donc que, dans ce cas "étude", la méthode LU appliquée à une matrice de diffusion de petite taille fournit une solution exacte à la précision machine. Ce résultat est cohérent avec le conditionnement modéré de la matrice et montre que l'implémentation de la factorisation et des phases de descente/remontée est correcte dans le cas scalaire bien conditionné.

Test général - matrice de diffusion, $N = 20$

Dans ce second test scalaire, nous considérons une matrice de diffusion tridiagonale de taille $N = 20$, de même structure que dans le cas « étude ». La matrice A est : Le

système est étudié avec cette fois un second membre différent, défini par

$$b = (1, 2, 3, \dots, 20)^T.$$

Avant la factorisation, le conditionnement en norme 2 de la matrice est évalué à

$$K_1(A) \approx 177,7694699965027.$$

Cette valeur est nettement plus élevée que dans le cas $N = 6$, ce qui indique une plus grande sensibilité aux erreurs d'arrondi.

La solution numérique calculée est

$$x \approx (73.33, 145.67, 216.00, 283.33, 346.67, 405.00, 457.33, 502.67\dots)^T.$$

Test "général" – matrice 2α , $N = 20$

Dans ce troisième test scalaire, nous considérons une matrice tridiagonale symétrique de taille $N = 20$ définie par une diagonale principale égale à 1,8 et des sous- et sur-diagonales égales à 1. Autrement dit,

$$A = \begin{pmatrix} 1.8 & 1 & 0 & \cdots & 0 \\ 1 & 1.8 & 1 & \ddots & \vdots \\ 0 & 1 & 1.8 & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & 1 \\ 0 & \cdots & 0 & 1 & 1.8 \end{pmatrix},$$

ce qui correspond à un opérateur de diffusion modifié où les interactions entre voisins sont de même ordre de grandeur que la diagonale. Le système étudié est

$$Ax = b, \quad \text{avec } b = (5, 10, 15, \dots, 100)^T.$$

Le conditionnement en norme 2 de la matrice est évalué à

$$K_1(A) = 17.183931150511100,$$

valeur nettement plus faible que dans le cas de diffusion classique de taille 20×20 , ce qui suggère une meilleure stabilité numérique à taille comparable.

Cas blocs

Test "étude" – matrice de Laplace par blocs, $N = 3$, $n = 3$

Dans ce premier test pour le cas par blocs, nous considérons la matrice tridiagonale par blocs correspondant à la discrétisation bidimensionnelle du Laplacien. Chaque bloc est de dimension 3×3 et la matrice complète A est donc de taille 9×9 . Sa structure est de la forme

$$A = \begin{pmatrix} B_1 & C_1 & 0 \\ A_2 & B_2 & C_2 \\ 0 & A_3 & B_3 \end{pmatrix},$$

avec $A_i = C_i = -I_3$ et $B_i = 4I_3 - T_3$, où T_3 est la matrice tridiagonale $(-1, 2, -1)$. La matrice complète A , affichée bloc par bloc, est donnée dans le fichier de sortie.

Le conditionnement en norme 2 de cette matrice est

$$K_1(A) = 5.828427124746190,$$

ce qui indique une très bonne stabilité numérique attendue.

La factorisation $A = LU$ préserve la structure par blocs. Les blocs diagonaux de L sont égaux à l'identité, tandis que les blocs sous-diagonaux sont non nuls :

$$L_2 = \begin{pmatrix} -0.26785714 & -0.07142857 & -0.01785714 \\ -0.07142857 & -0.28571429 & -0.07142857 \\ -0.01785714 & -0.073142857 & -0.26785714 \end{pmatrix},$$

$$L_3 = \begin{pmatrix} -0.29482402 & -0.09316770 & -0.02815735 \\ -0.09316770 & -0.32298137 & -0.09316770 \\ -0.02815735 & -0.09316770 & -0.29482402 \end{pmatrix}.$$

Les blocs diagonaux de U obtenus numériquement sont

$$U_1 = \begin{pmatrix} 4 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix}, \quad U_2 = \begin{pmatrix} 3.73214286 & -1.07142857 & -0.01785714 \\ -1.07142857 & 3.71428571 & -1.07142857 \\ -0.01785714 & -1.07142857 & 3.73214286 \end{pmatrix},$$

$$U_3 = \begin{pmatrix} 3.70517598 & -1.09316770 & -0.02815735 \\ -1.09316770 & 3.67701863 & -1.09316770 \\ -0.02815735 & -1.09316770 & 3.70517598 \end{pmatrix}.$$

Enfin, les blocs hors diagonale C_1 et C_2

Le vecteur b choisi vérifie $b = A \cdot \mathbf{1}$, de sorte que la solution exacte est

$$X_{\text{exact}} = (1, 1, 1, 1, 1, 1, 1, 1)^T.$$

La résolution numérique donne exactement la même solution par blocs :

$$X_1 = (1, 1, 1)^T, \quad X_2 = (1, 1, 1)^T, \quad X_3 = (1, 1, 1)^T.$$

L'erreur euclidienne obtenue est

$$\|X - 1\|_2 = 0.0000000000000000,$$

ce qui montre que la solution est exacte à la précision machine.

Test général – matrice de Laplace par blocs, $N = 4$, $n = 4$

Ce second test par blocs considère la même matrice de Laplace 2D que dans le cas "étude", mais de plus grande taille : chaque bloc est de dimension 4×4 et la matrice complète A est donc de taille 16×16 . Sa structure par blocs est de la forme

$$A = \begin{pmatrix} B_1 & C_1 & 0 & 0 \\ A_2 & B_2 & C_2 & 0 \\ 0 & A_3 & B_3 & C_3 \\ 0 & 0 & A_4 & B_4 \end{pmatrix},$$

avec $A_i = C_i = -I_4$ et $B_i = 4I_4 - T_4$, où T_4 est la matrice tridiagonale $(-1, 2, -1)$ de dimension 4. La matrice complète A , affichée bloc par bloc, est donnée dans le fichier de sortie.

Le conditionnement en norme 2 de cette matrice est

$$K_1(A) = 9.472135954999580,$$

ce qui reste une valeur relativement faible et laisse a priori espérer une bonne stabilité numérique.

Problème rencontré pendant la factorisation. Contrairement au test précédent ($N = 3$, $n = 3$), le fichier de résultats montre que la factorisation LU ne s'affiche pas correctement : les matrices L , U ainsi que le vecteur solution X ne sont pas imprimés. Cela indique que le programme a été interrompu au cours de l'exécution, ou qu'une écriture

prématurée du fichier de sortie a eu lieu avant la fin de la factorisation.

Ce comportement est typique d'une instabilité numérique dans la factorisation LU sans pivot. En effet : les blocs L_i sont calculés à partir de l'inverse des blocs diagonaux U_{i-1} , si un bloc diagonal U_i devient numériquement quasi singulier, son inverse contient des valeurs très grandes, ces valeurs se propagent dans les blocs suivants de L et U et la descente ou la remontée devient alors impossible ou conduit à des valeurs non définies.

Ce phénomène n'apparaissait pas dans le cas $N = 3$, $n = 3$, car la taille plus petite rendait la factorisation beaucoup plus stable.

Ce test général met en évidence un point important : la méthode LU par blocs fonctionne parfaitement lorsque les blocs restent bien conditionnés au cours de la factorisation, elle peut cependant échouer numériquement même si la matrice globale est bien conditionnée pour rendre la méthode fiable en grande dimension, il serait souhaitable d'introduire un pivot partiel par blocs ou d'envisager l'utilisation de méthodes itératives. Ce cas illustre donc les limites pratiques de la factorisation LU sans pivot dans le cadre des matrices tridiagonales par blocs.

Conclusion

Dans ce projet, nous avons étudié la résolution numérique de systèmes linéaires de grande dimension, d'abord en utilisant des matrices tridiagonales scalaires, puis en généralisant la méthode au cas plus complexe des matrices tridiagonales par blocs. L'objectif principal était de mettre en oeuvre la factorisation LU , d'évaluer ses performances numériques et d'analyser l'influence du conditionnement des matrices sur la qualité de la solution.

Les expériences menées dans la partie scalaire ont montré que la méthode LU est très efficace lorsque la matrice est bien conditionnée. Pour les tests "étude", les solutions obtenues étaient exactes à la précision machine, et l'erreur était nulle après la descente et la remontée. Cependant, les tests généraux ont révélé une réalité importante : une matrice de conditionnement modéré peut déjà produire des facteurs L et U dont certains coefficients deviennent très grands en valeur absolue.

La deuxième partie du projet, consacrée aux matrices par blocs, a confirmé et amplifié ces observations. Pour le test "étude". La factorisation par blocs a été parfaitement stable et a permis d'obtenir la solution exacte. En revanche, pour le test "général", bien que la matrice globale soit correctement conditionnée, la factorisation LU a échoué numériquement : la méthode s'est bloquée avant la remontée, et aucune solution n'a pu être affichée. Ce résultat met en évidence un phénomène essentiel : le conditionnement d'une matrice ne suffit pas à garantir la stabilité de la factorisation.

Pour améliorer la robustesse de la méthode, plusieurs pistes sont envisageables : introduire un pivot partiel ou un pivot par blocs afin d'éviter les divisions par des éléments proches de zéro ; utiliser des méthodes itératives mieux adaptées ou bien exploiter des préconditionneurs afin de réduire le conditionnement des systèmes à résoudre.

Bilans personnalisés

NGUYEN Manh Hung : Ce projet a été une occasion importante de mettre en pratique les méthodes numériques vues en cours, mais aussi d'apprendre à gérer un travail informatique plus complexe. Au début du projet, j'avais des difficultés à comprendre la structure des matrices tridiagonales par blocs et à traduire les expressions mathématiques en code C. J'ai donc commencé par travailler sur la partie scalaire, ce qui m'a permis de bien comprendre la factorisation LU . Ensuite, j'ai pris en charge principalement l'implémentation de la version "général" ainsi que les tests numériques. J'ai également écrit le script permettant de calculer le conditionnement des matrices et les outils pour ma-

nipuler les matrices. Grâce à cette partie, j'ai compris que le conditionnement n'est pas seulement un concept théorique, mais un élément essentiel pour analyser la qualité des résultats numériques.

NGO Tan Minh Duy : Ce projet m'a permis de consolider mes connaissances en programmation C et en calcul scientifique. J'ai d'abord travaillé sur la gestion des fichiers de données, l'allocation dynamique de la mémoire et l'affichage des résultats sous forme de matrices et de vecteurs. Cette étape était indispensable pour rendre le programme clair, lisible et exploitable pendant les tests. J'ai ensuite contribué à l'implémentation de la factorisation LU par blocs. C'était la partie la plus technique du projet, car il fallait manipuler des blocs $n \times n$ plutôt que des scalaires, et s'assurer que les opérations matricielles étaient correctement appliquées. Même si la version "général" a montré des instabilités numériques, j'ai appris à identifier l'origine du problème et à analyser les limites de la méthode.

Références

- [1] P. Lascaux et R. Théodor, *Analyse numérique matricielle appliquée à l'art de l'ingénieur*, Tome 1, 2^e édition, Masson, 1993.